

W7 PRACTICE

BLABLA PROVIDERS & REPOSITORIES

Learning objectives

- ✓ **Refactor** the BlaBla project using the **Provider State Management**
- ✓ **Implement** a **Local preferences repository** to persist the history of ride preferences
- ✓ **Implement** a **Firestore Realtime database** to provide the locations

How to start?

- ✓ [Copy the START CODE](#) into a new repository (BLA-PROJECT-W7) and push it

BLA-200- Start Code Week 7

- ✓ Each task of this practice shall be related to commit(s) including the **tasks ID**.

BLA-201 – Add equals method to RidePref model

How to submit?

- ✓ Once finished, submit on MS Team
 - Your GitHub repository URL
 - This document if needed



This practice focusses more **on clean code structure** rather than Flutter technical skills.
We will evaluate how well **you follow the coding conventions**, how you name your class, variables
etc.

BLA-201 - Rides Preferences with Provider

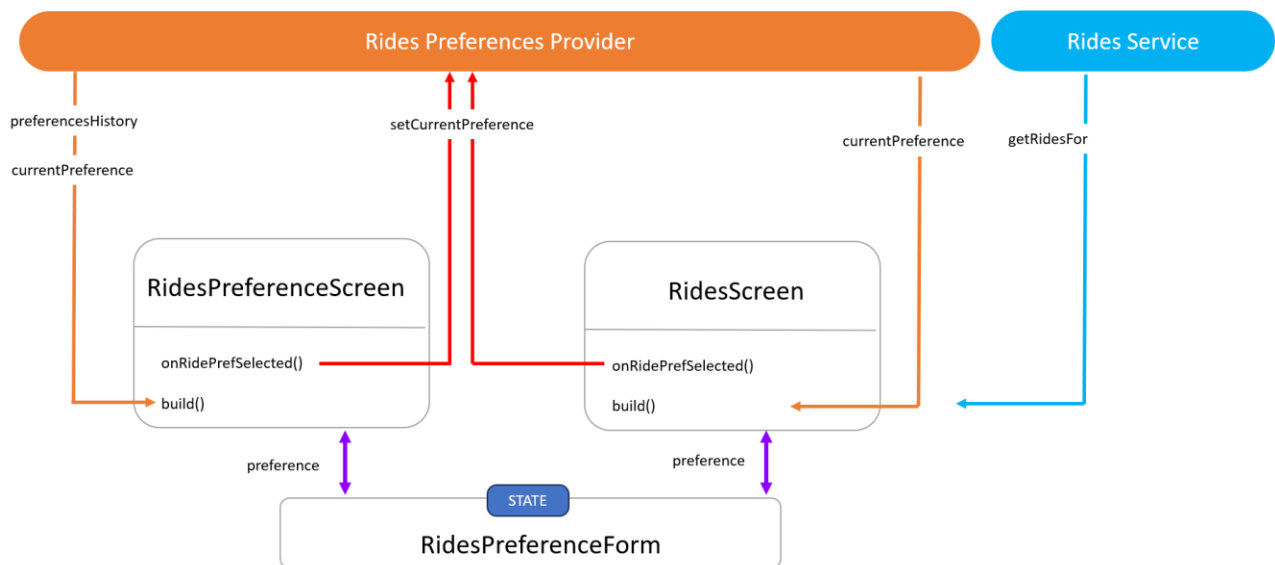
[Get the START CODE](#)

The goal of this task is to switch from the previous RidesPreferences Service to a Provider.

- The 2 screens (rides preferences and rides) **interact with the provider**
- Other widgets are given data (data drilling)

Why do we change?

- A provider (*Value Notifier*) is the place to store the local data needed by the screens.
- A provider will handle the widget notifications
- At the end, the objective is to simplify the code handled by the different views



Why do you need to update?

We suggest the following step.

We advise you to go step by step and **understand what you are doing and why**.

CLASS	TODO
MODELS	
RidePreference	Implement the operator == and hash Code methods <i>See Location class for example</i> <i>Why? To be able to compare Ride preferences</i>
SERVICES	
RidePrefService	To be removed, as we will use the Provider now
PROVIDERS	
RidesPreferencesProvider	- This provider requires a RidePreferencesRepository - The provider manages

	- the current preference (can be null) - the past preferences (empty by default) - <i>Note : for now, we fetch the list of past preferences in the constructor</i> - The provider provides 3 methods: - <code>currentPreference</code> Get the current preference (can be null) - <code>setCurrentPreference</code> Set the current preference - <code>preferencesHistory</code> Get the past preferences (from the newest to the lasted) - setCurrentPreference method shall process as follow: 1- We process only if the new preference is not equal to the current one 2- We first update the current preference 3- We then update the history <i>All preferences in history shall be exclusive (different) !</i> 4- We finally notify the listeners
SERVICES	
RidesPreferenceScreen	- Shall be a stateless widget now, as we are using the RidesPreferencesProvider <i>Warning : the context (Build Context) needs to be passed as parameters to the methods (ex onRidePrefSelected)</i> - build() - Watch the RidesPreferencesProvider - get the current preference - get the history preferences - onRidePrefSelected() - Read the RidesPreferencesProvider - Call the provider setCurrentPreference
RidesScreen	- Shall be a stateless widget now, as we are using the RidesPreferencesProvider <i>Warning : the context (Build Context) needs to be passed as parameters to the methods (ex onRidePrefSelected)</i> - build() - Watch the RidesPreferencesProvider - get the current preference - get the list of available rides regarding the current ride preference - onRidePrefSelected() - Read the RidesPreferencesProvider

	- Call the provider setCurrentPreference
RidePrefForm	A small refresh issue ! - initState() only runs once when the widget is first created. - If RidePrefForm is not fully recreated, initState() does not re-run - So initState() is not called when we update the view on provider notification - overwrite the method: didUpdateWidget - perform the same code as done in initState
MAIN	
main	- Create a MultiProvider - Create a ChangeNotifierProvider to expose the RidesPreferencesProvider

Here is the start code for the provider, to guide you on the way:

```
class RidesPreferencesProvider extends ChangeNotifier {

  RidePreference? _currentPreference;
  List<RidePreference> _pastPreferences = [];

  final RidePreferencesRepository repository;

  RidesPreferencesProvider({required this.repository}) {
    // For now past preferences are fetched only 1 time
    // Your code
  }

  RidePreference? get currentPreference => _currentPreference;

  void setCurrentPreference(RidePreference pref) {
    // Your code
  }

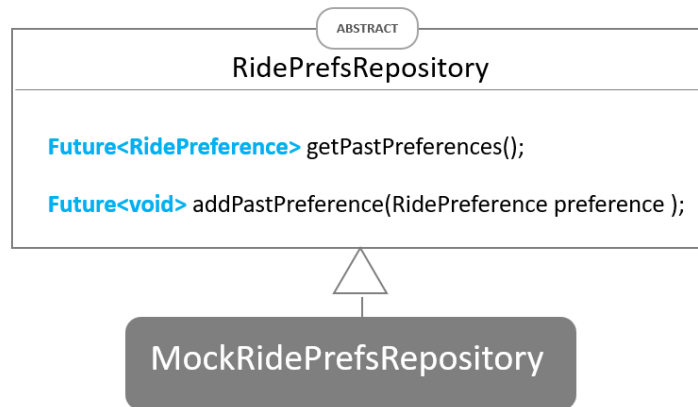
  void _addPreference(RidePreference preference) {
    // Your code
  }

  // History is returned from newest to oldest preference
  List<RidePreference> get preferencesHistory =>
    _pastPreferences.reversed.toList();
}
```

BLA-202 – Repository and Provider with **Future<T>**

The goal of this task is to handle synchronous calls and Futures.

For now we want to test our async calls with the mock repo, to ensure we can react to every states.



STEP 1 - Update the repositories

- **Abstract** repository:

```
Future<List<RidePreference>> getPastPreferences()
Future<void> addPreference(RidePreference preference)
```

- **Mock** repository: wait 2 seconds on both operations (get and add)

STEP 2 - Add the Async Value data structure

- Copy/ Paste the **AsyncValue** class used in the previous practice, inside the **provider** folder.

STEP 3 - Update RidesPreferencesProvider

- Wrap the pastPreferences with a **AsyncValue** :

```
late AsyncValue<List<RidePreference>> pastPreferences;
```

- Create a **fetchPastPreferences()** method to handle Future states (*loading, success, error*)

```

// 1- Handle loading
pastPreferences = AsyncValue.loading();
notifyListeners();

try {
    // 2 Fetch data
    List<RidePreference> pastPrefs = await repository.getPastPreferences();

    // 3 Handle success
    pastPreferences = AsyncValue.success(pastPrefs);

    // 4 Handle error
} catch (error) {
    pastPreferences = AsyncValue.error(error);
}

notifyListeners();
}

```

- Update the `_addPreference` method to handle the asynchronous call :

We can consider 2 approaches to handle a change in the database:

First approach :

- | | |
|---|--|
| 1- Call the repository <code>addPreference()</code> | <code>await repository.addPreference(pref);</code> |
| 2- Fetch again the data from the repository | <code>_fetchPastPreferences();</code> |

Second approach :

- | | |
|---|--|
| 1- Call the repository <code>addPreference()</code> | <code>await repository.addPreference(pref);</code> |
| 2- Update the provider cache | <code>pastPreferences.data.add(newPref)</code> |

Which approach will you choose? Why ?

STEP 4 - Update `RidesPreferencesScreen`

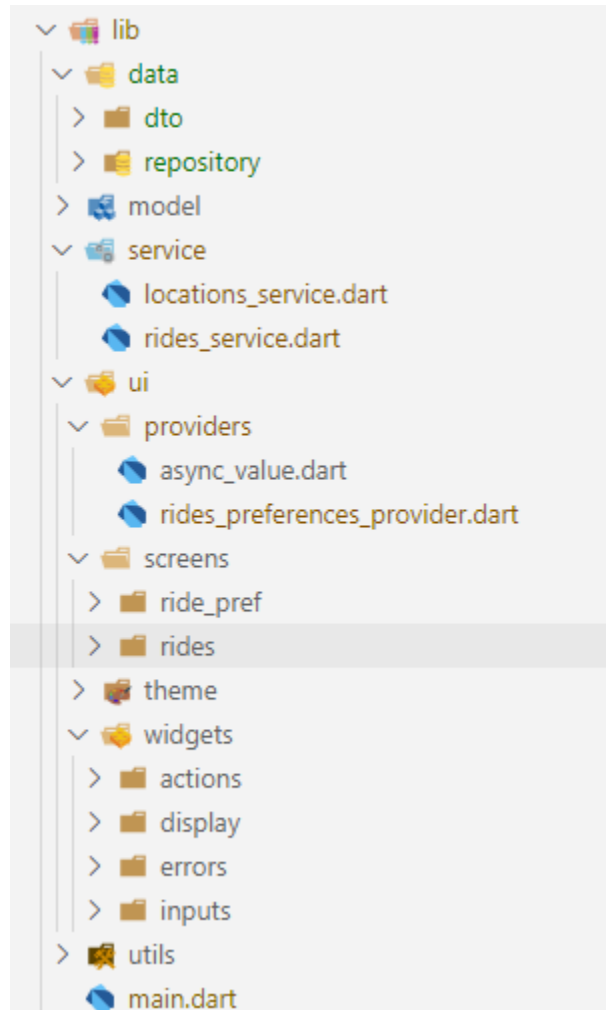
Finally update the `build()` code in the **`RidesPreferencesScreen`** screen to handle the states of the `pastPreferences` :

- If the state is loading, return a **`BlaError`** with a 'loading' message
- If the state is error, return a **`BlaError`** with a 'No connection. Try later' message
- If the state is success, display the screen as normal

BLA-203– Refactor your project structure

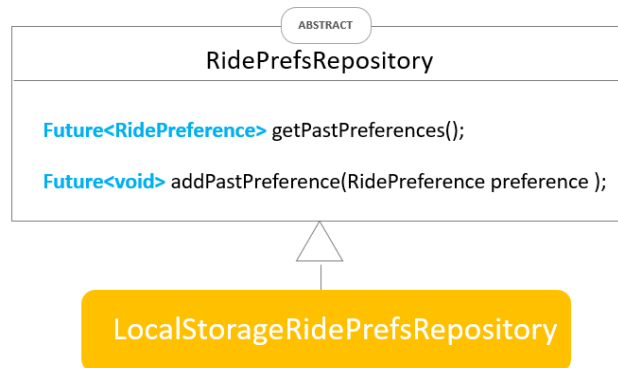
The goal of this task is to re-organize your BlaBla project around 3 main layers:

DATA	Repositories, and some other data structures needed for databases
MODEL / SERVICE	Business of the application, regardless of the data storage and UI
UI	UI providers, screens, widgets



BLA-204 – Implement the Local Storage repository

The goal of this task is to implement a repository to **store rides preference locally on the smartphone**.



https://pub.dev/packages/shared_preferences

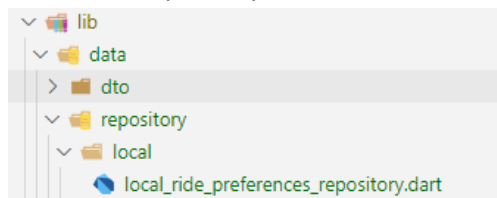
SharedPreferences in Flutter is a simple key-value storage system used to persist small amounts of data locally on a device.

STEP 1 - Add the SharedPreferences to the project

```
flutter pub add shared_preferences
```

STEP 2 - Implement the LocalRidePreferencesRepository

- Create the class LocalRidePreferencesRepository which extends PreferencesRepository



- Create a **key** to store past preferences

```
static const String _preferencesKey = "ride_preferences";
```

- Implement **getPastPreferences**

```
// Get SharedPreferences instance
final prefs = await SharedPreferences.getInstance();

// Get the string list from the key
final prefsList = prefs.getStringList(_preferencesKey) ?? [];

// Convert the string list to a list of RidePreferences - Using map()
return prefsList.map((json) => RidePreferenceDto.fromJson(jsonDecode(json))).toList();
```

- Implement **addPastPreference**

- 1 Call getPastPreferences
- 2 Add the new preference

3 Save the new list as a string list

```
// Save the new list as a string list
await prefs.setStringList(
  _preferencesKey,
  preferences.map((pref) => jsonEncode(RidePreferenceDto.toJson(pref))).toList(),
);
```

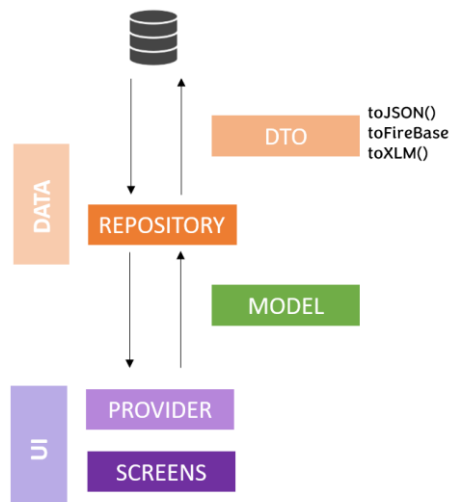
STEP 2 - Implement the Data Transfers methods

We need to serialize the objects Location and RidePreference into JSON format.

Read first the documentation to understand JSON serialization in Dart

<https://docs.flutter.dev/data-and-backend/serialization/json>

To keep our project clean, we will put the code related to JSON conversion into specific classed named DTO.



We keep the code to convert the model to the database types into DTO classes

- Create 2 classes to convert Location and RidePreference from and to JSON:
 - o LocationDto
 - o RidePreferenceDto
- On each class, implement the static methods for model to JSON and JSON to model conversions:

```

class LocationDto {
    static Map<String, dynamic> toJson(Location model) {
        return {
            'name': model.name,
            'country': model.country.name, // Store enum as string
        };
    }

    static Location fromJson(Map<String, dynamic> json) {
        return Location(
            name: json['name'],
            country: countryFromString(json['country']),
        );
    }

    static Country countryFromString(String value) {
        return Country.values.firstWhere(
            (e) => e.name == value,
            orElse: () => throw ArgumentError('Invalid country: $value'),
        );
    }
}

```

Example of JSON conversion method for the Location class

- Implement **getPastPreferences**

```

// Get SharedPreferences instance
final prefs = await SharedPreferences.getInstance();

// Get the string list from the key
final prefsList = prefs.getStringList(_preferencesKey) ?? [];

// Convert the string list to a list of RidePreferences - Using map()
return prefsList.map((json) => RidePreferenceDto.fromJson(jsonDecode(json))).toList();

```

STEP 3 - Test your repository

Well done! At this step you should be able to test the application and check the history of entered preferences stored locally on the mobile.

BLA-205 – **BONUS** - Connecting to Firebase Realtime Database via REST API

The goal of this task is to implement a repository to **store locations on Firebase**, using a **Real time database** and a REST API connection.

<https://firebase.google.com/docs/database/rest/start>

<https://firebase.google.com/docs/reference/rest/database>

Step 1: Create a Firebase Project

1. Go to Firebase Console and create a new project.
2. Navigate to **Build > Realtime Database** and click **Create Database**.
3. Choose the **Start in test mode** option for easy access during development.

Step 2: Add Sample Data in Firebase

1. Go to the **Realtime Database** section.
2. Click on the **Rules** tab and set the following to allow public read access (for testing purposes):

```
{
  "rules": {
    ".read": true,
    ".write": false
  }
}
```

3. In the **Data** tab, add the 3 sample locations

Step 3: Implement the Repository

Create a the `location_firebase_repository.dart` implementing an abstract `location_repository.dart`

```
Future<List<Location>> getLocations()
```

Step 4: Test the repository

As done on previous sections, you will have to create a Provider and update the code to handle asynchronous states in the LocationPicker.